

agentic-ai-azure-week08-rag-grounding

Week 8 - Memory, State & Grounding (RAG)

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week08-rag-grounding' - Week 8 - Memory, State & Grounding (RAG). Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Build retrieval-grounded agents
- Make knowledge permission-aware (security trimming)
- Cite sources; say 'I don't know' when ungrounded

Notes

[Learning goal] Walk through these talking points:

- Build retrieval-grounded agents
- Make knowledge permission-aware (security trimming)
- Cite sources; say 'I don't know' when ungrounded

Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Clinical Policy Assistant (healthcare)
- Clinicians query care protocols
- Answers trimmed by role/department; sources cited

Notes

[Enterprise use case] Walk through these talking points:

- Clinical Policy Assistant (healthcare)
- Clinicians query care protocols
- Answers trimmed by role/department; sources cited

Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Ingest -> chunk -> embed -> index (Azure AI Search)
- Hybrid search (keyword + vector, RRF)
- Security trimming via OData ACL filter
- Grounded generation + 'I don't know' guardrail

Notes

[Architecture / flow] Walk through these talking points:

- Ingest -> chunk -> embed -> index (Azure AI Search)
- Hybrid search (keyword + vector, RRF)
- Security trimming via OData ACL filter
- Grounded generation + 'I don't know' guardrail

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- Real Search backend + setup_search.py ingestion
- Embeddings via Foundry; HNSW vector profile
- Mock backend mirrors retrieval + trimming offline

Notes

[Key concepts] Walk through these talking points:

- Real Search backend + setup_search.py ingestion
- Embeddings via Foundry; HNSW vector profile
- Mock backend mirrors retrieval + trimming offline

Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/policy/ask
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/policy/ask
- Runs in MOCK mode - no Azure/keys needed

Pause for questions before moving on.

Slide 7 - Architect's takeaways

- Security trimming is a first-class requirement (PHI/PII)
- Retrieval quality: chunking, hybrid, reranking
- Grounding vs. hallucination guardrails

Notes

[Architect's takeaways] Walk through these talking points:

- Security trimming is a first-class requirement (PHI/PII)
- Retrieval quality: chunking, hybrid, reranking
- Grounding vs. hallucination guardrails

Discussion: when would you NOT choose this approach?